

K-Fold_RoBERTa_Multiclassification

May 9, 2026

```
[6]: # =====  
# RoBERTa (base)  
# FIXED VERSION (Path + Loading Issues Solved)  
# =====  
  
import os  
import random  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from tqdm import tqdm, trange  
from sklearn.model_selection import train_test_split, StratifiedKFold  
from sklearn.metrics import classification_report, confusion_matrix  
from imblearn.over_sampling import RandomOverSampler  
  
import torch  
from torch.utils.data import Dataset, DataLoader  
from torch.optim import AdamW  
from transformers import (  
    RobertaTokenizerFast,  
    RobertaForSequenceClassification,  
    get_linear_schedule_with_warmup  
)  
  
# --- Reproducibility ---  
SEED = 42  
random.seed(SEED)  
np.random.seed(SEED)  
torch.manual_seed(SEED)  
  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
print(f"Using device: {device}")  
  
# =====  
# FIX: USE CORRECT MODEL PATH OR ONLINE MODEL  
# =====
```

```

USE_LOCAL = False    # CHANGE TO True if you want local model

# --- FIXED: Correct Indentation ---
if USE_LOCAL:
    dataset_path = 'annotated_dataset_for_multi_classifications.csv' # Define the dataset path

    df = pd.read_csv(dataset_path, encoding='utf-8-sig') # Use the defined dataset path

    print("Using LOCAL model")
    print("Exists:", os.path.exists(LOCAL_MODEL_PATH))
    print("Files:", os.listdir(LOCAL_MODEL_PATH))

    tokenizer = RobertaTokenizerFast.from_pretrained(LOCAL_MODEL_PATH)
    MODEL_NAME = LOCAL_MODEL_PATH
else:
    print("Using ONLINE model (recommended)")
    tokenizer = RobertaTokenizerFast.from_pretrained("roberta-base")
    MODEL_NAME = "roberta-base"

# =====
# --- Load Dataset ---
# =====
dataset_path = 'annotated_dataset_for_multi_classifications.csv' # Define the dataset path

df = pd.read_csv(dataset_path, encoding='utf-8-sig') # Use the defined dataset path

print(" Dataset loaded successfully")
print("Columns in dataset:", df.columns.tolist())

# =====
# --- Label Mapping ---
# =====

y_dict = {
    'functionality issue': 0,
    'user experience issue': 1,
    'performance issue': 2,
    'stability issue': 3,
    'bug': 4,
    'device issue': 5,
    'customer support issue': 6,
    'privacy issue': 7,

```

```

        'user interface issue': 8,
        'compatibility issue': 9,
        'installation issue': 10,
        'security issue': 11,
        'network issue': 12
    }

df['category'] = df['category'].str.strip().str.lower()
df = df[df['category'].isin(y_dict)]
df['label_id'] = df['category'].map(y_dict)

print("Dataset size:", len(df))

# =====
# --- Train/Test Split ---
# =====

texts = df['Base_Reviews'].values
y_int = df['label_id'].values

X_train_raw, X_test_raw, y_train_raw, y_test = train_test_split(
    texts, y_int, test_size=0.15, random_state=SEED, stratify=y_int
)

print(f"Train: {len(X_train_raw)}, Test: {len(X_test_raw)}")

# =====
# --- Tokenization ---
# =====

def tokenize_texts(texts, max_len):
    return tokenizer(
        list(map(str, texts)),
        padding='max_length',
        truncation=True,
        max_length=max_len,
        return_tensors='pt'
    )

# =====
# --- Dataset Class ---
# =====

class TextDS(Dataset):
    def __init__(self, toks, labels):
        self.ids = toks['input_ids']
        self.msk = toks['attention_mask']

```

```

        self.y = torch.tensor(labels, dtype=torch.long)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, i):
        return {
            'input_ids': self.ids[i],
            'attention_mask': self.msk[i],
            'labels': self.y[i]
        }

# =====
# --- Training Function ---
# =====

def train_one_run(train_loader, val_loader, epochs, lr):

    model = RobertaForSequenceClassification.from_pretrained(
        MODEL_NAME,
        num_labels=len(y_dict)
    ).to(device)

    optimizer = AdamW(model.parameters(), lr=lr)
    scheduler = get_linear_schedule_with_warmup(
        optimizer, 0, len(train_loader) * epochs
    )

    best_acc = 0
    best_state = None

    for ep in range(epochs):
        print(f"\nEpoch {ep+1}/{epochs}")

        # Train
        model.train()
        total_loss, correct = 0, 0

        for batch in tqdm(train_loader):
            optimizer.zero_grad()

            outputs = model(
                batch['input_ids'].to(device),
                attention_mask=batch['attention_mask'].to(device),
                labels=batch['labels'].to(device)
            )

```

```

        loss = outputs.loss
        loss.backward()
        optimizer.step()
        scheduler.step()

        total_loss += loss.item()
        preds = outputs.logits.argmax(1)
        correct += (preds == batch['labels'].to(device)).sum().item()

train_acc = correct / len(train_loader.dataset)

# Validation
model.eval()
val_correct = 0

with torch.no_grad():
    for batch in val_loader:
        outputs = model(
            batch['input_ids'].to(device),
            attention_mask=batch['attention_mask'].to(device)
        )
        preds = outputs.logits.argmax(1)
        val_correct += (preds == batch['labels'].to(device)).sum().
↪item()

val_acc = val_correct / len(val_loader.dataset)

print(f"Train Acc: {train_acc:.4f} | Val Acc: {val_acc:.4f}")

if val_acc > best_acc:
    best_acc = val_acc
    best_state = model.state_dict()

model.load_state_dict(best_state)
return model

# =====
# --- Hyperparameters ---
# =====

BS = 8
EPOCHS = 8
MAXLEN = 128
LR = 3e-5

# =====
# --- 5-Fold CV ---

```

```

# =====

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

for fold, (tr_idx, val_idx) in enumerate(skf.split(X_train_raw, y_train_raw),
↳1):
    print(f"\nFold {fold}")

    X_tr, X_val = X_train_raw[tr_idx], X_train_raw[val_idx]
    y_tr, y_val = y_train_raw[tr_idx], y_train_raw[val_idx]

    ros = RandomOverSampler(random_state=SEED)
    X_tr, y_tr = ros.fit_resample(X_tr.reshape(-1,1), y_tr)
    X_tr = X_tr.flatten()

    train_ds = TextDS(tokenize_texts(X_tr, MAXLEN), y_tr)
    val_ds = TextDS(tokenize_texts(X_val, MAXLEN), y_val)

    model = train_one_run(
        DataLoader(train_ds, batch_size=BS, shuffle=True),
        DataLoader(val_ds, batch_size=BS),
        EPOCHS, LR
    )

# =====
# --- Final Training ---
# =====

ros = RandomOverSampler(random_state=SEED)
X_train_res, y_train_res = ros.fit_resample(X_train_raw.reshape(-1,1),
↳y_train_raw)
X_train_res = X_train_res.flatten()

train_ds = TextDS(tokenize_texts(X_train_res, MAXLEN), y_train_res)
test_ds = TextDS(tokenize_texts(X_test_raw, MAXLEN), y_test)

model = train_one_run(
    DataLoader(train_ds, batch_size=BS, shuffle=True),
    DataLoader(test_ds, batch_size=BS),
    EPOCHS, LR
)

# =====
# --- Final Evaluation ---
# =====

model.eval()

```

```

y_true, y_pred = [], []

with torch.no_grad():
    for batch in DataLoader(test_ds, batch_size=BS):
        outputs = model(
            batch['input_ids'].to(device),
            attention_mask=batch['attention_mask'].to(device)
        )
        preds = outputs.logits.argmax(1)
        y_true.extend(batch['labels'].numpy())
        y_pred.extend(preds.cpu().numpy())

print("\nClassification Report:\n")
print(classification_report(y_true, y_pred))

# =====
# --- Confusion Matrix ---
# =====

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()

```

Using device: cpu

Using ONLINE model (recommended)

Dataset loaded successfully

Columns in dataset: ['Rating_Star', 'Headings', 'Issue', 'Base_Reviews',
'category', 'Issue Details']

Dataset size: 16034

Train: 13628, Test: 2406

Fold 1

Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized:

['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias',
'classifier.out_proj.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Epoch 1/8

huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid deadlocks...

To disable this warning, you can either:

- Avoid using `tokenizers` before the fork if possible

- Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

100%| | 5872/5872 [2:24:34<00:00, 1.48s/it]

Train Acc: 0.8196 | Val Acc: 0.8302

Epoch 2/8

100%| | 5872/5872 [2:42:53<00:00, 1.66s/it]

Train Acc: 0.9475 | Val Acc: 0.8313

Epoch 3/8

100%| | 5872/5872 [2:16:51<00:00, 1.40s/it]

Train Acc: 0.9666 | Val Acc: 0.8668

Epoch 4/8

100%| | 5872/5872 [2:18:50<00:00, 1.42s/it]

Train Acc: 0.9736 | Val Acc: 0.8635

Epoch 5/8

100%| | 5872/5872 [2:18:49<00:00, 1.42s/it]

Train Acc: 0.9775 | Val Acc: 0.8701

Epoch 6/8

100%| | 5872/5872 [2:18:03<00:00, 1.41s/it]

Train Acc: 0.9803 | Val Acc: 0.8811

Epoch 7/8

100%| | 5872/5872 [2:19:31<00:00, 1.43s/it]

Train Acc: 0.9813 | Val Acc: 0.8822

Epoch 8/8

100%| | 5872/5872 [2:19:07<00:00, 1.42s/it]

Train Acc: 0.9827 | Val Acc: 0.8870

Fold 2

Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized:

['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias', 'classifier.out_proj.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Epoch 1/8

100%| | 5872/5872 [2:23:14<00:00, 1.46s/it]

Train Acc: 0.8272 | Val Acc: 0.8496

Epoch 2/8

100%| | 5872/5872 [2:20:05<00:00, 1.43s/it]

Train Acc: 0.9500 | Val Acc: 0.8485

Epoch 3/8

100%| | 5872/5872 [2:23:42<00:00, 1.47s/it]

Train Acc: 0.9658 | Val Acc: 0.8859

Epoch 4/8

100%| | 5872/5872 [2:29:42<00:00, 1.53s/it]

Train Acc: 0.9733 | Val Acc: 0.8885

Epoch 5/8

100%| | 5872/5872 [2:24:34<00:00, 1.48s/it]

Train Acc: 0.9779 | Val Acc: 0.8995

Epoch 6/8

100%| | 5872/5872 [2:23:24<00:00, 1.47s/it]

Train Acc: 0.9805 | Val Acc: 0.8940

Epoch 7/8

100%| | 5872/5872 [2:40:04<00:00, 1.64s/it]

Train Acc: 0.9818 | Val Acc: 0.9057

Epoch 8/8

100%| | 5872/5872 [2:22:33<00:00, 1.46s/it]

Train Acc: 0.9829 | Val Acc: 0.9039

Fold 3

Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized:

['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias', 'classifier.out_proj.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Epoch 1/8

100%| | 5870/5870 [2:23:00<00:00, 1.46s/it]

Train Acc: 0.8265 | Val Acc: 0.7704

Epoch 2/8

100%| | 5870/5870 [2:25:16<00:00, 1.48s/it]

Train Acc: 0.9497 | Val Acc: 0.8584

Epoch 3/8

100%| | 5870/5870 [2:21:03<00:00, 1.44s/it]

Train Acc: 0.9667 | Val Acc: 0.8723

Epoch 4/8

100%| | 5870/5870 [2:19:52<00:00, 1.43s/it]

Train Acc: 0.9733 | Val Acc: 0.8936

Epoch 5/8

100%| | 5870/5870 [2:18:12<00:00, 1.41s/it]

Train Acc: 0.9762 | Val Acc: 0.8859

Epoch 6/8

100%| | 5870/5870 [2:21:33<00:00, 1.45s/it]

Train Acc: 0.9802 | Val Acc: 0.8910

Epoch 7/8

100%| | 5870/5870 [2:18:16<00:00, 1.41s/it]

Train Acc: 0.9820 | Val Acc: 0.9013

Epoch 8/8

100%| | 5870/5870 [2:20:25<00:00, 1.44s/it]

Train Acc: 0.9830 | Val Acc: 0.8988

Fold 4

Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized:
['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias',

`'classifier.out_proj.weight']`

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Epoch 1/8

100%| | 5872/5872 [2:37:12<00:00, 1.61s/it]

Train Acc: 0.8318 | Val Acc: 0.8433

Epoch 2/8

100%| | 5872/5872 [2:30:40<00:00, 1.54s/it]

Train Acc: 0.9517 | Val Acc: 0.8591

Epoch 3/8

100%| | 5872/5872 [2:30:20<00:00, 1.54s/it]

Train Acc: 0.9682 | Val Acc: 0.8598

Epoch 4/8

100%| | 5872/5872 [2:29:56<00:00, 1.53s/it]

Train Acc: 0.9744 | Val Acc: 0.8829

Epoch 5/8

100%| | 5872/5872 [2:29:39<00:00, 1.53s/it]

Train Acc: 0.9778 | Val Acc: 0.8892

Epoch 6/8

100%| | 5872/5872 [2:27:37<00:00, 1.51s/it]

Train Acc: 0.9805 | Val Acc: 0.8939

Epoch 7/8

100%| | 5872/5872 [2:24:11<00:00, 1.47s/it]

Train Acc: 0.9833 | Val Acc: 0.8958

Epoch 8/8

100%| | 5872/5872 [2:25:47<00:00, 1.49s/it]

Train Acc: 0.9839 | Val Acc: 0.8998

Fold 5

Some weights of `RobertaForSequenceClassification` were not initialized from the model checkpoint at `roberta-base` and are newly initialized:

```
['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias',  
'classifier.out_proj.weight']
```

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Epoch 1/8

```
100%|          | 5872/5872 [2:26:38<00:00, 1.50s/it]
```

Train Acc: 0.8263 | Val Acc: 0.8294

Epoch 2/8

```
100%|          | 5872/5872 [2:25:32<00:00, 1.49s/it]
```

Train Acc: 0.9516 | Val Acc: 0.8661

Epoch 3/8

```
100%|          | 5872/5872 [2:24:00<00:00, 1.47s/it]
```

Train Acc: 0.9681 | Val Acc: 0.8730

Epoch 4/8

```
100%|          | 5872/5872 [2:27:01<00:00, 1.50s/it]
```

Train Acc: 0.9750 | Val Acc: 0.8910

Epoch 5/8

```
100%|          | 5872/5872 [2:23:42<00:00, 1.47s/it]
```

Train Acc: 0.9785 | Val Acc: 0.8921

Epoch 6/8

```
100%|          | 5872/5872 [2:26:29<00:00, 1.50s/it]
```

Train Acc: 0.9813 | Val Acc: 0.8987

Epoch 7/8

```
100%|          | 5872/5872 [2:22:08<00:00, 1.45s/it]
```

Train Acc: 0.9828 | Val Acc: 0.8972

Epoch 8/8

```
100%|          | 5872/5872 [2:30:44<00:00, 1.54s/it]
```

Train Acc: 0.9844 | Val Acc: 0.8987

Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized:

```
['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias',
```

```
'classifier.out_proj.weight']
```

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Epoch 1/8

100%| | 7339/7339 [2:54:41<00:00, 1.43s/it]

Train Acc: 0.7910 | Val Acc: 0.8678

Epoch 2/8

100%| | 7339/7339 [2:58:19<00:00, 1.46s/it]

Train Acc: 0.9505 | Val Acc: 0.9061

Epoch 3/8

100%| | 7339/7339 [3:02:53<00:00, 1.50s/it]

Train Acc: 0.9660 | Val Acc: 0.9115

Epoch 4/8

100%| | 7339/7339 [3:03:19<00:00, 1.50s/it]

Train Acc: 0.9736 | Val Acc: 0.9023

Epoch 5/8

100%| | 7339/7339 [3:01:42<00:00, 1.49s/it]

Train Acc: 0.9771 | Val Acc: 0.9198

Epoch 6/8

100%| | 7339/7339 [3:02:02<00:00, 1.49s/it]

Train Acc: 0.9791 | Val Acc: 0.9260

Epoch 7/8

100%| | 7339/7339 [3:01:54<00:00, 1.49s/it]

Train Acc: 0.9810 | Val Acc: 0.9244

Epoch 8/8

100%| | 7339/7339 [3:03:11<00:00, 1.50s/it]

```

[2]: # =====
# HYPERPARAMETERS (added to fix NameError)
# =====
BEST_BS = 16          # Best batch size
BEST_LR = 2e-5        # Best learning rate
BEST_EPOCHS = 3       # Best number of epochs
SEED = 42             # Random seed for reproducibility

# =====
# FINAL MODEL: Retrain on FULL OVERSAMPLED TRAIN (85%)
# =====
print(f"\n Retraining final BERT on FULL TRAIN (85%)...")

# Oversample FULL TRAIN only
ros_final = RandomOverSampler(random_state=SEED)
X_train_res, y_train_res = ros_final.fit_resample(X_train_raw.reshape(-1, 1),
    ↪ y_train_raw)
X_train_res = X_train_res.flatten()

# Tokenize
tok_train_final = tokenize_texts(X_train_res, MAXLEN)
tok_test_final = tokenize_texts(X_test_raw, MAXLEN)

train_ds_final = TextDS(tok_train_final, y_train_res)
test_ds_final = TextDS(tok_test_final, y_test)

train_loader_final = DataLoader(train_ds_final, batch_size=BEST_BS,
    ↪ shuffle=True)
test_loader_final = DataLoader(test_ds_final, batch_size=BEST_BS)

# Train final model
final_model, _, final_history = train_one_run(
    train_loader_final,
    DataLoader(
        TextDS(tokenize_texts(X_train_raw[:500], MAXLEN), y_train_raw[:500]),
        batch_size=BEST_BS
    ), # small internal val for early stopping
    epochs=BEST_EPOCHS,
    lr=BEST_LR
)

# Save
SAVE_DIR = './bert_best'
os.makedirs(SAVE_DIR, exist_ok=True)

```

```

final_model.save_pretrained(SAVE_DIR)
tokenizer.save_pretrained(SAVE_DIR)
print(f"\n Saved BERT model to: {SAVE_DIR}")

# =====
# FINAL EVALUATION ON TEST SET (15%)
# =====
test_acc, test_loss, y_true_test, y_pred_test = evaluate_on_loader(final_model,
    ↪test_loader_final)
print(f"\n FINAL TEST ACCURACY (Held-Out, Natural Distribution): {test_acc:.
    ↪4f}")

print("\n=== FINAL CLASSIFICATION REPORT (TEST SET) ===")
print(classification_report(y_true_test, y_pred_test,
    ↪target_names=class_labels, digits=4))

# Confusion Matrix
cm = confusion_matrix(y_true_test, y_pred_test)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
    ↪xticklabels=class_labels, yticklabels=class_labels)
plt.title('BERT Confusion Matrix - Final Test')
plt.xlabel('Predicted'); plt.ylabel('True')
plt.tight_layout()
plt.savefig('BERT_ConfusionMatrix_Test.png', dpi=300, bbox_inches='tight')
plt.show()

# Learning Curves
plt.figure(figsize=(13, 5))
epochs = range(1, len(final_history['train_loss']) + 1)

plt.subplot(1, 2, 1)
plt.plot(epochs, final_history['train_loss'], 'b-o', label='Train Loss')
plt.plot(epochs, final_history['val_loss'], 'r-s', label='Val Loss')
plt.title('Loss'); plt.xlabel('Epoch'); plt.legend(); plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(epochs, final_history['train_acc'], 'b-o', label='Train Acc')
plt.plot(epochs, final_history['val_acc'], 'r-s', label='Val Acc')
plt.title('Accuracy'); plt.xlabel('Epoch'); plt.legend(); plt.grid(True)

plt.suptitle('BERT Final Training')
plt.tight_layout()
plt.savefig('BERT_LearningCurves.png', dpi=300, bbox_inches='tight')
plt.show()

```